Mock 1 Easy 01 | Retail Order Quality - PySpark

SUBMISSION TEMPLATE

Difficulty	Marks	Tasks	Working budget
Easy	10	4	17 minutes (Easy)

Objective

Prepare valid retail orders, calculate sales, and reconcile category totals.

Evidence rules - read before starting

There is NO one-task/one-screenshot mandate and NO maximum screenshot count. Provide as many clear taskwise images as the rubric requires. If both implementation and result are assessed, include BOTH. A long SQL/PySpark/pipeline definition may be split into Code_A, Code_B and Code_C screenshots, plus separate Result_A / Result_B screenshots. Images must be legible, not tiny stitched collages. A UI/configuration screenshot is acceptable when the task is menu-based. Never rerun a mutation only to capture evidence; use the original executed code and saved result. The evaluator awards marks only from the submitted visible evidence; merely supplying a workspace link is not a substitute.

Environment and required names

Use your existing assigned Unity Catalog. Reuse schema mock1 for all four questions. Create the schema if missing; do not create a catalog, storage credential, external location, or an unrelated warehouse. Each question creates ONLY its own EXTERNAL volume using the instructor-supplied complete ADLS URI beneath the approved external location. Upload source CSVs inside input/. Managed tables and SDP output must be in schema-managed Delta storage, not inside volume files.

Item	Required value
Catalog	Existing assigned trainee catalog
Schema	mock1
Question external volume	m1_e01_files
Assigned relative ADLS path	mock1/e01/files/ under trainee root
Input directory	/Volumes/<trainee_catalog>/mock1/m1_e01_files/input/
Required saved object(s)	m1_e01_retail_orders

Key terms and formulas

- Valid order: quantity must be greater than zero AND unit_price must be greater than zero.

- sales_amount = quantity x unit_price, presented to two decimal places.

Dataset description

File(s): retail_orders.csv. Header present; use the supplied data types. Blank values, if any, represent missing values. Do not rename files.

Column	Type	Meaning
order_id	STRING	Unique order
category	STRING	Product category
quantity	INTEGER	Ordered units
unit_price	DECIMAL(12,2)	Unit price

Trainee: _____Soumiya Ayyappan_____ Candidate ID: _____ Catalog: _____

Date: _____ Notebook / SQL editor / pipeline / dashboard link: _____

Task requirements and screenshot placeholders

Task 1. Prepare governed source and load orders | 2 marks

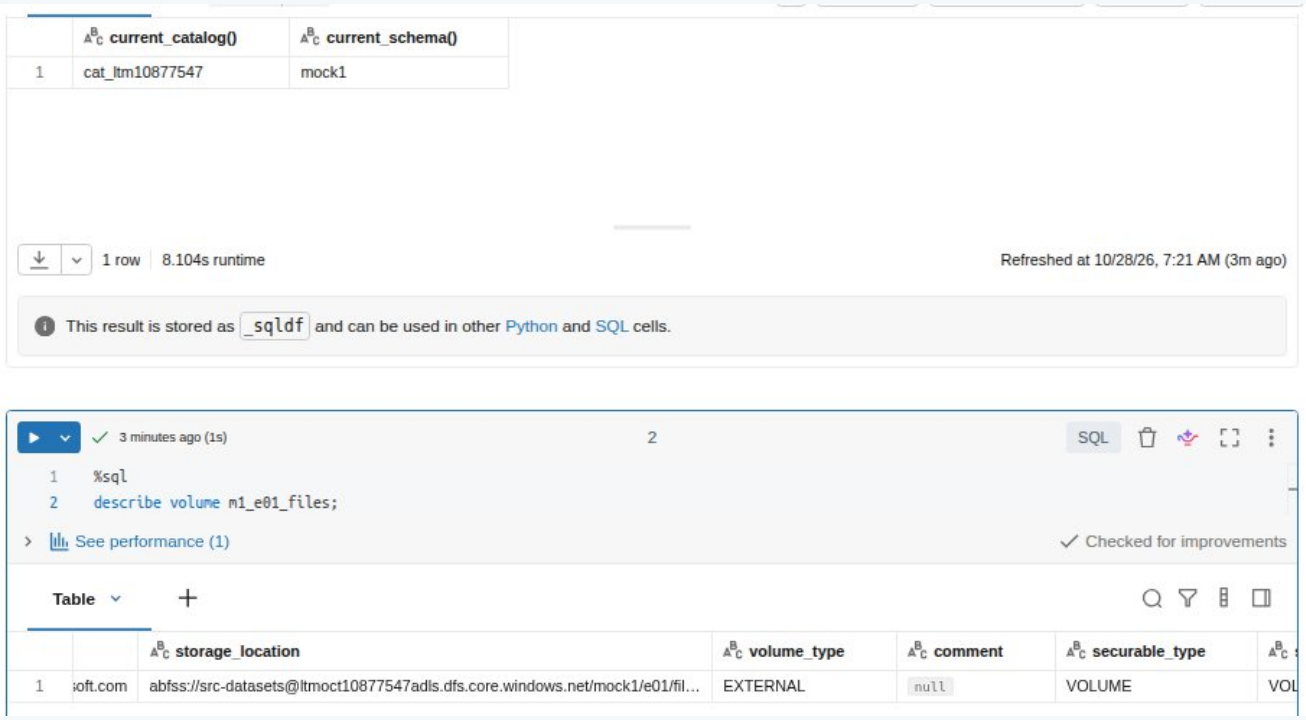
A. Setup UI: EXTERNAL volume, full assigned URI, and input filename.

B. Code: volume source path and typed CSV read.

C. Result: eight source records and four column names.

Use descriptive filenames such as M1_E01_T01_Code_A.png, M1_E01_T01_Result_A.png, or M1_E01_T01_UI_A.png. Add _B or _C if code/output does not fit. These are examples, not a maximum number.

Paste screenshot(s) - Setup UI: _____



	current_catalog()	current_schema()
1	cat_ltm10877547	mock1

1 row | 8.104s runtime

Refreshed at 10/28/26, 7:21 AM (3m ago)

This result is stored as _sqldf and can be used in other Python and SQL cells.

3 minutes ago (1s)

2

SQL

```
%sql
describe volume m1_e01_files;
```

See performance (1)

Checked for improvements

Table

		storage_location	volume_type	comment	securable_type	
1	soft.com	abfss://src-datasets@ltmoct10877547adls.dfs.core.windows.net/mock1/e01/fil…	EXTERNAL	null	VOLUME	VOL

Paste screenshot(s) - Code: _____

```python
# Volume source path
source_path = "/Volumes/cat_ltm10877547/mock1/m1_e01_files/input/E01_retail_orders.csv"

# Explicit schema
orders_schema = StructType([
    StructField("order_id", StringType(), True),     # Unique order
    StructField("category", StringType(), True),     # Product category
    StructField("quantity", IntegerType(), True),    # Ordered units
    StructField("unit_price", DecimalType(12, 2), True)  # Unit price
])

# Typed CSV read
orders_df = (spark.read
    .format("csv")
    .option("header", "true")
    .schema(orders_schema)
    .load(source_path))

print("Source records:", orders_df.count())
print("Column names:", orders_df.columns)
orders_df.printSchema()
display(orders_df)
```

> See performance (2)

Paste screenshot(s) - Result: _____

```
Source records: 8
Column names: ['order_id', 'category', 'quantity', 'unit_price']
root
 |-- order_id: string (nullable = true)
 |-- category: string (nullable = true)
 |-- quantity: integer (nullable = true)
 |-- unit_price: decimal(12,2) (nullable = true)
```

Table +

	order_id	category	quantity	unit_price
1	O101	Electronics	2	300.00
2	O102	Grocery	3	50.00
3	O103	Electronics	1	200.00
4	O104	Clothing	2	100.00
5	O105	Grocery	0	40.00
6	O106	Clothing	1	-10.00
7	O107	Grocery	4	25.00
8	O108	Electronics	1	100.00

Additional screenshot(s) for this task (if required): _____

Evidence reference / saved object: _____

Task 2. Filter valid orders | 3 marks

A. Code: both filter conditions and the input DataFrame.

B. Result: six retained IDs and two rejected IDs (or equivalent count and rejected rows).

Use descriptive filenames such as M1_E01_T02_Code_A.png, M1_E01_T02_Result_A.png, or M1_E01_T02_UI_A.png. Add _B or _C if code/output does not fit. These are examples, not a maximum number.

Paste screenshot(s) - Code: _____

Just now (2s) 4

```
from pyspark.sql.functions import col

# Filter conditions: quantity > 0 AND unit_price > 0
filtered_df = orders_df.filter((col("quantity") > 0) & (col("unit_price") > 0))
rejected_df = orders_df.subtract(filtered_df)

retained_count = filtered_df.count()
rejected_count = rejected_df.count()

print(f"Retained records: {retained_count}")
print(f"Rejected records: {rejected_count}")

print("\n--- Retained rows ---")
display(filtered_df)

print("\n--- Rejected rows ---")
display(rejected_df)
```

Paste screenshot(s) - Result: _____

> 2 dataframes: filtered_df, rejected_df

```
Retained records: 6
Rejected records: 2

--- Retained rows ---
```

Table +

	order_id	category	quantity	unit_price
1	O101	Electronics	2	300.00
2	O102	Grocery	3	50.00
3	O103	Electronics	1	200.00
4	O104	Clothing	2	100.00
5	O107	Grocery	4	25.00
6	O108	Electronics	1	100.00

```
--- Rejected rows ---
```

Table +

	order_id	category	quantity	unit_price
1	O106	Clothing	1	-10.00
2	O105	Grocery	0	40.00

Additional screenshot(s) for this task (if required): _____

Evidence reference / saved object: _____

Task 3. Calculate and persist sales | 3 marks

A. Code: calculated column and managed Delta persistence.

B. Result: all six order sales and named saved table.

Use descriptive filenames such as M1_E01_T03_Code_A.png, M1_E01_T03_Result_A.png, or M1_E01_T03_UI_A.png. Add _B or _C if code/output does not fit. These are examples, not a maximum number.

Paste screenshot(s) - Code: _____

Just now (5s) 5

```python
from pyspark.sql.functions import col, round as spark_round

# Add calculated column: total_sales = quantity * unit_price
sales_df = filtered_df.withColumn("total_sales", spark_round(col("quantity") * col("unit_price"), 2))

print(f"Records with total_sales: {sales_df.count()}")
display(sales_df)

# Persist as managed Delta table
table_name = "cat_ltm10877547.mock1.m1_e01_retail_orders"
sales_df.write.format("delta").mode("overwrite").saveAsTable(table_name)
print(f"Saved managed Delta table: {table_name}")
```

Paste screenshot(s) - Result: _____

Records with total_sales: 6

Table +

	order_id	category	quantity	unit_price	total_sales
1	O101	Electronics	2	300.00	600.00
2	O102	Grocery	3	50.00	150.00
3	O103	Electronics	1	200.00	200.00
4	O104	Clothing	2	100.00	200.00
5	O107	Grocery	4	25.00	100.00
6	O108	Electronics	1	100.00	100.00

6 rows | 4.896s runtime

Saved managed Delta table: cat_ltm10877547.mock1.m1_e01_retail_orders

Additional screenshot(s) for this task (if required): _____

Evidence reference / saved object: _____

Task 4. Reconcile category sales | 2 marks

A. Code: grouping, summation and descending sort from saved table.

B. Result: three category totals and grand total 1350.00.

Use descriptive filenames such as M1_E01_T04_Code_A.png, M1_E01_T04_Result_A.png, or M1_E01_T04_UI_A.png. Add _B or _C if code/output does not fit. These are examples, not a maximum number.

Paste screenshot(s) - Code: _____

```
Just now (2s)                                                    6

1  from pyspark.sql.functions import col, sum as spark_sum, lit
2
3  # Read from the saved managed Delta table
4  table_name = "cat_ltm10877547.mock1.m1_e01_retail_orders"
5  orders_df = spark.read.table(table_name)
6
7  # Group by category, sum total_sales, sort descending
8  category_totals_df = (
9      orders_df
10     .groupBy("category")
11     .agg(spark_sum("total_sales").alias("total_sales"))
12     .orderBy(col("total_sales").desc())
13 )
14
15 print("--- Category Totals (descending) ---")
16 display(category_totals_df)
17
18 # Grand total
19 grand_total = orders_df.agg(spark_sum("total_sales").alias("grand_total")).collect()[0]["grand_total"]
20 print(f"\nGrand Total: {grand_total}")

> See performance (2)
```

Paste screenshot(s) - Result: _____

> 2 dataframes: category_totals_df, orders_df

```
--- Category Totals (descending) ---
```

Table +

	category	total_sales
1	Electronics	900.00
2	Grocery	250.00
3	Clothing	200.00

3 rows | 1.592s runtime

```
Grand Total: 1350.00
```

Additional screenshot(s) for this task (if required): _____

Evidence reference / saved object: _____

Evaluation rubric and evidence map

Task	Criterion	Marks	Evidence
1	Correct assigned external volume, URI and upload	1	Code / UI / Result as specified in Task 1
1	Typed CSV read and eight-record evidence	1	Code / UI / Result as specified in Task 1
2	Both mandatory validation conditions implemented	2	Code / UI / Result as specified in Task 2
2	Correct six retained and two excluded records	1	Code / UI / Result as specified in Task 2
3	Correct sales formula, typed numeric result and managed table	2	Code / UI / Result as specified in Task 3
3	Correct saved six-row sales detail	1	Code / UI / Result as specified in Task 3
4	Correct group-by and source table	1	Code / UI / Result as specified in Task 4
4	Correct ordered category and grand totals	1	Code / UI / Result as specified in Task 4

Total marks: 10. Task count: 4. Screenshot count: variable; provide enough legible evidence to cover every rubric criterion.

Submission requirements

Complete the matching submission template with task-specific screenshot(s) showing every assessed code/configuration and output item. Multiple screenshots per task are allowed and sometimes necessary. Code-only evidence does not prove output; result-only evidence does not prove implementation when the rubric assesses both. Do not submit a generic solution script in place of evidence. Keep your original notebook, Delta objects, pipeline, dashboard and version history accessible. No extra hidden tests or additional report are required.